Unsupervised Learning

- Unsupervised Learning is a type of machine learning where the algorithm learns from unlabeled data meaning the model tries to find patterns, groupings, or structures in the data without being told the correct answer.

- The model explores the data and identifies hidden structures without human supervision.

1- Clustering

Groups similar data points together.

Example: Grouping customers by purchasing behavior.

Algorithms:

- K-Means
- DBSCAN
- Hierarchical Clustering

2- Dimensionality Reduction

Reduces the number of features while preserving important information.

Example: Visualizing high-dimensional data in 2D.

Algorithms:

- PCA (Principal Component Analysis)
- t-SNE
- UMAP

3-Anomaly Detection

Detects data points that don't conform to expected patterns.

Example: Fraud detection, rare disease diagnosis.

4- Association Rules

Discovers interesting relationships between variables in large datasets.

Example: Market basket analysis (people who buy bread often buy butter).

Algorithm: Apriori

1. K-Means Clustering

- Groups data into **K distinct clusters**.
- Minimizes the distance between points and their assigned cluster center (centroid).
- You must choose K in advance.

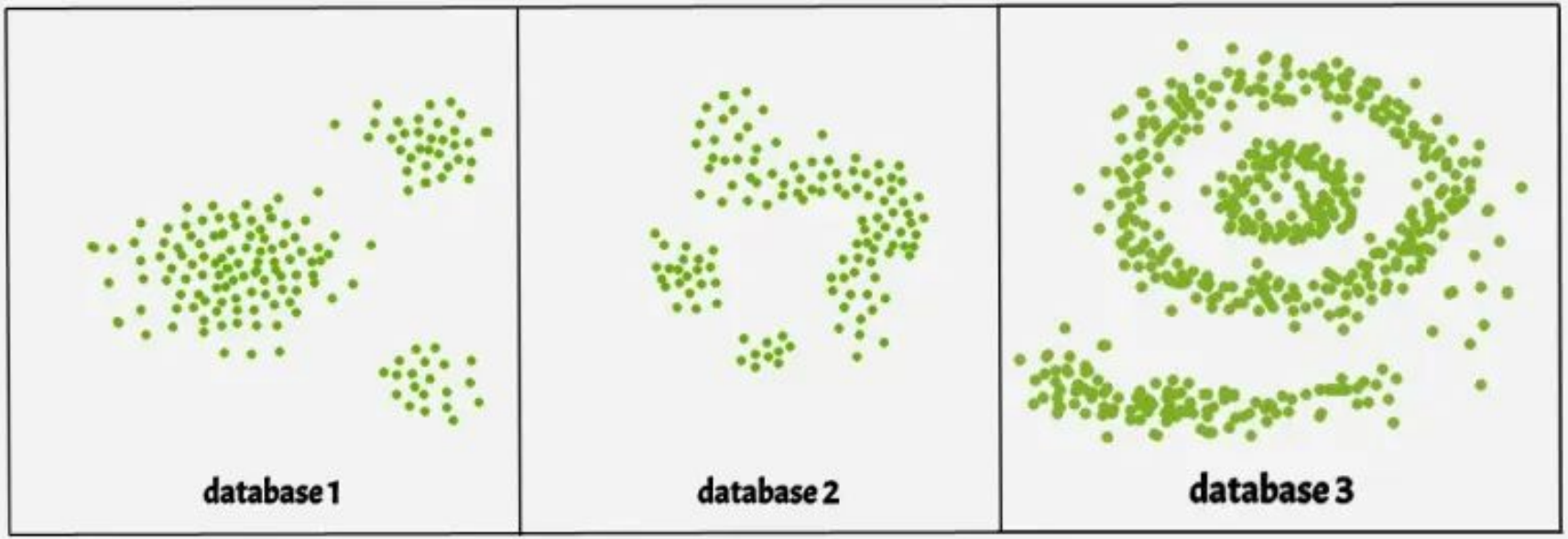
2- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is a density-based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space.

It identifies clusters as dense regions in the data space separated by areas of lower density.

Unlike K-Means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN perform well in handling real-world data irregularities such as:

- **Arbitrary-Shaped Clusters**: Clusters can take any shape not just circular or convex.
- **Noise and Outliers**: It effectively identifies and handles noise points without assigning them to any cluster.


database 1
database 2
database 3

Key Parameters in DBSCAN

1. eps: This defines the radius of the neighborhood around a data point. If the distance between two points is less than or equal to eps they are considered neighbors.

A common method to determine eps is by analyzing the k-distance graph. Choosing the right eps is important:

- If eps is too small most points will be classified as noise.
- If eps is too large clusters may merge and the algorithm may fail to distinguish between them.

2. MinPts: This is the minimum number of points required within the **eps** radius to form a dense region. A general rule of thumb is to set MinPts >= D+1 where **D** is the number of dimensions in the dataset.

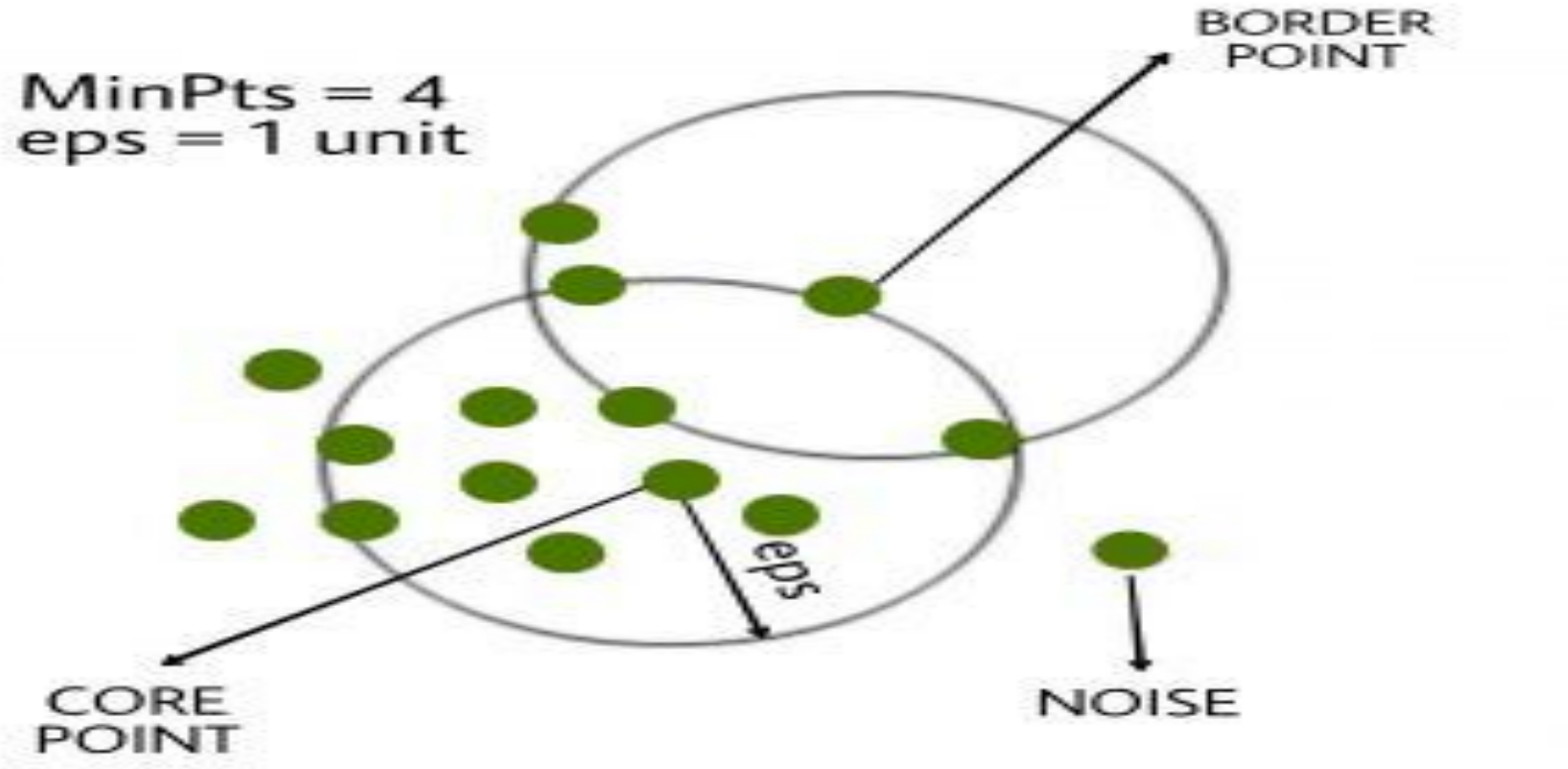
*For most cases a minimum value of **MinPts = 3** is recommended*

How Does DBSCAN Work?

DBSCAN works by categorizing data points into three types:

1. Core points which have a sufficient number of neighbors within a specified radius (eplison)
2. Border points which are near core points but lack enough neighbors to be core points themselves
3. Noise points which do not belong to any cluster.

By iteratively expanding clusters from core points and connecting density-reachable points, DBSCAN forms clusters without relying on rigid assumptions about their shape or size.


MinPts = 4
eps = 1 unit
BORDER POINT
eps
CORE POINT
NOISE

Evaluation Metrics For DBSCAN Algorithm In Machine Learning :

1-Silhouette Score:

The Silhouette Score (often called the Silhouette Coefficient) is not a clustering algorithm, but rather a metric used to evaluate clustering quality.

A validation metric used to measure how well each data point fits within its cluster compared to other clusters.

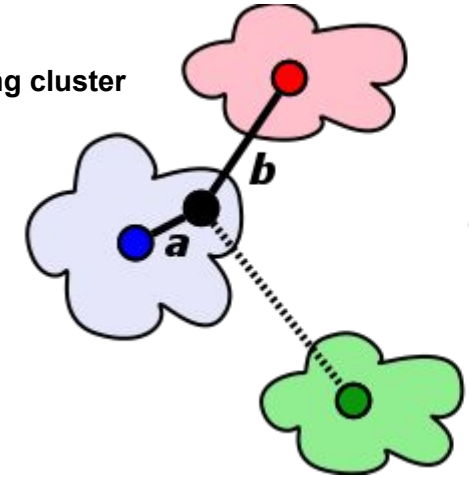
Values range from **-1 to 1**:

- **+1** → well matched to its own cluster, far from others (good).
- **0** → on or very close to the decision boundary between clusters.
- **-1** → wrongly assigned to the wrong cluster (bad).

How it's calculated for a point ?

- **a** = average distance to other points in the **same cluster**
- **b** = average distance to points in the **nearest neighboring cluster**

Then the silhouette score for a single point is:



$$SSI_i = \frac{b_i - a_i}{max(a_i, b_i)}$$

Hierarchical clustering

is used to group similar data points together based on their similarity creating a **hierarchy or tree-like structure**.

The key idea is to begin with each data point as its own separate cluster and then progressively merge or split them based on their similarity.

*Imagine you have four fruits with different weights: an **apple (100g)**, **a banana (120g)**, **a cherry (50g) and a grape (30g)**. Hierarchical clustering starts by treating each **fruit as its own group**.*

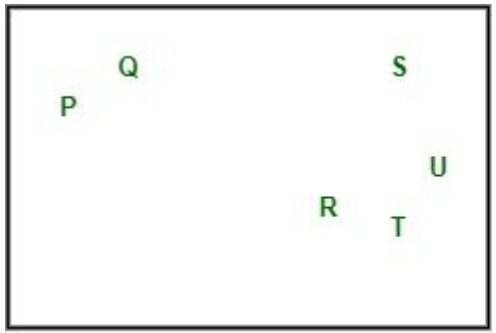
- It then merges the closest groups based on their weights.
- First the cherry and grape are grouped together because they are the lightest.
- Next the apple and banana are grouped together.

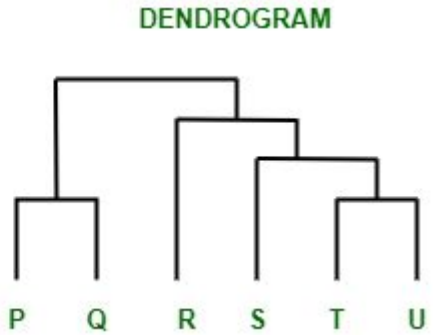
Finally all the fruits are merged into one large group, showing how hierarchical clustering progressively combines the most similar data points.

Dendogram

A **dendrogram** is like a family tree for clusters. It shows how individual data points or groups of data merge together.

The bottom shows each data point as its own group, and as you move up, similar groups are combined.



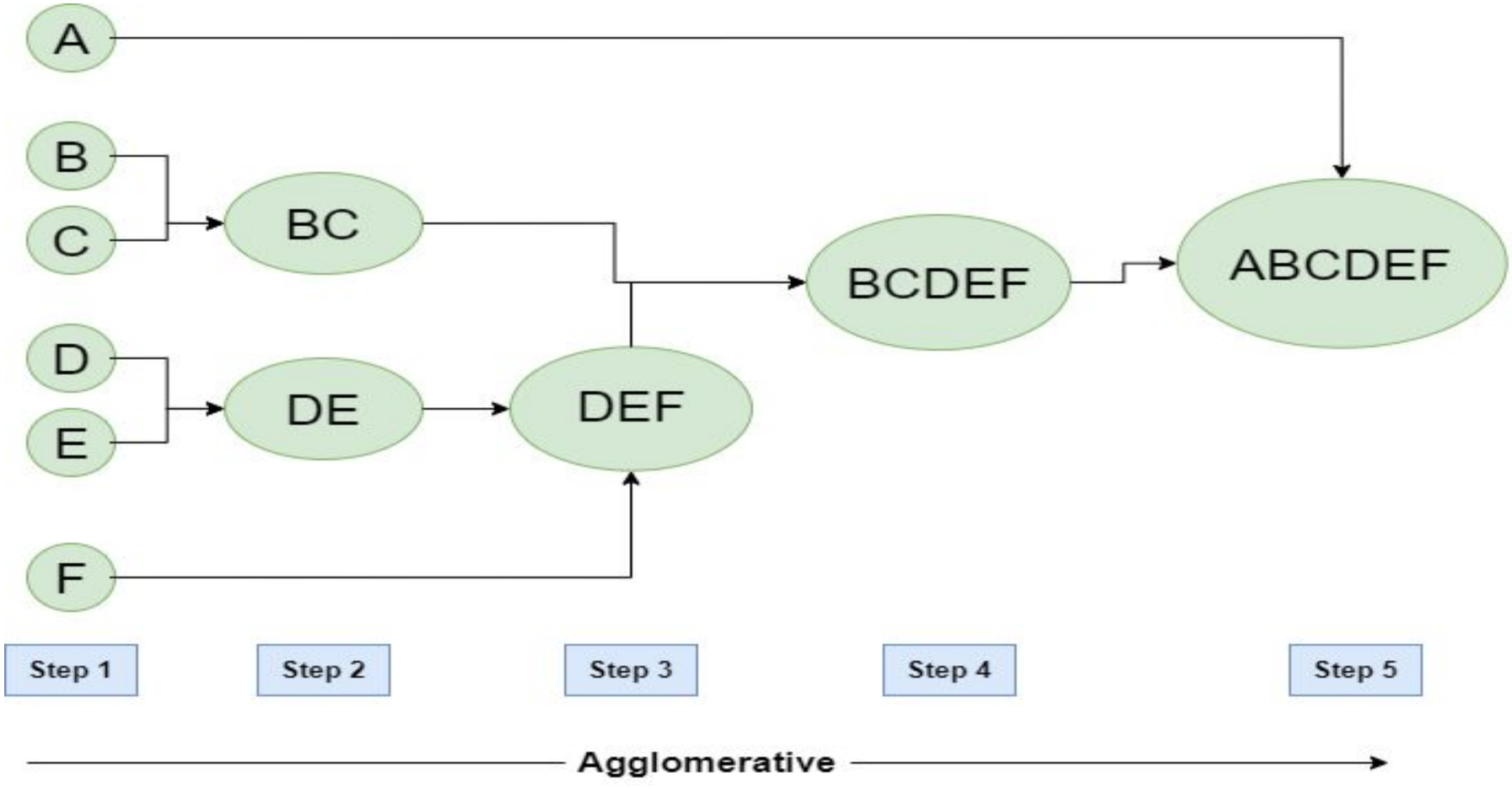


Types of Hierarchical Clustering

1. Agglomerative Clustering
2. Divisive clustering

Hierarchical Agglomerative Clustering

- It is also known as the bottom-up approach or hierarchical agglomerative clustering (HAC).
- Unlike flat clustering hierarchical clustering provides a structured way to group data.
- This clustering algorithm does not require us to prespecify the number of clusters. Bottom-up algorithms treat each data as a singleton cluster at the outset and then successively agglomerate pairs of clusters until all clusters have been merged into a single cluster that contains all data.

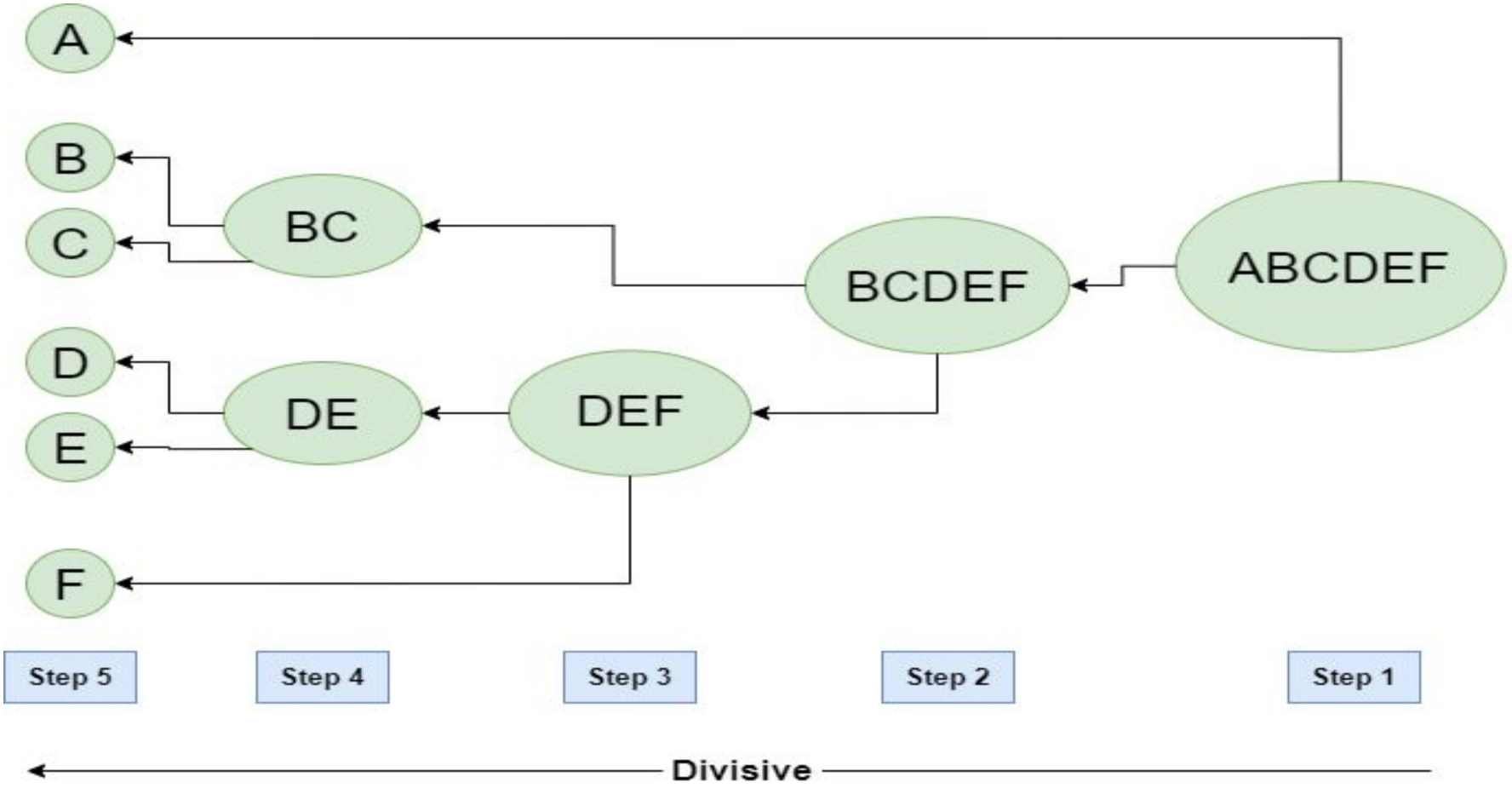

A
B
C
BC
D
E
DE
DEF
F
BCDEF
ABCDEF
Step 1
Step 2
Step 3
Step 4
Step 5
Agglomerative

1. **Start with individual points**: Each data point is its own cluster. For example if you have 5 data points you start with 5 clusters each containing just one data point.
2. **Calculate distances between clusters**: Calculate the distance between every pair of clusters. Initially since each cluster has one point this is the distance between the two data points.
3. **Merge the closest clusters**: Identify the two clusters with the smallest distance and merge them into a single cluster.
4. **Update distance matrix**: After merging you now have one less cluster. Recalculate the distances between the new cluster and the remaining clusters.
5. **Repeat steps 3 and 4**: Keep merging the closest clusters and updating the distance matrix until you have only one cluster left.
6. **Create a dendrogram**: As the process continues you can visualize the merging of clusters using a tree-like diagram called a **dendrogram**. It shows the hierarchy of how clusters are merged.

Hierarchical Divisive clustering

It is also known as a top-down approach.

This algorithm also does not require to prespecify the number of clusters. Top-down clustering requires a method for splitting a cluster that contains the whole data and proceeds by splitting clusters recursively until individual data have been split into singleton clusters.


A
B
C
BC
BCDEF
ABCDEF
D
E
DE
DEF
F
Step 5
Step 4
Step 3
Step 2
Step 1
Divisive

1. **Start with all data points in one cluster**: Treat the entire dataset as a single large cluster.
2. **Split the cluster**: Divide the cluster into two smaller clusters. The division is typically done by finding the two most dissimilar points in the cluster and using them to separate the data into two parts.
3. **Repeat the process**: For each of the new clusters, repeat the splitting process:
    1. Choose the cluster with the most dissimilar points.
    2. Split it again into two smaller clusters.
4. **Stop when each data point is in its own cluster**: Continue this process until every data point is its own cluster, or the stopping condition (such as a predefined number of clusters) is met.

Computing Distance Matrix

While merging two clusters we **check the distance between two every pair of clusters and merge the pair with the least distance/most similarity**.

There are different ways of defining Inter Cluster distance/similarity. Some of them are:

1. **Min Distance:** Find the minimum distance between any two points of the cluster.
2. **Max Distance:** Find the maximum distance between any two points of the cluster.
3. **Group Average:** Find the average distance between every two points of the clusters.
4. **Ward's Method:** The similarity of two clusters is based on the increase in squared error when two clusters are merged.
5.

2 - Dimensionality Reduction

Reduces the number of features while preserving important information.
Visualizing high-dimensional data in 2D.
Algorithms:

- PCA (Principal Component Analysis

PCA (Principal Component Analysis) is a dimensionality reduction technique used in data analysis and machine learning.

It helps you to reduce the number of features in a dataset while keeping the most important information. It changes your original features into new features these new features don't overlap with each other and the first few keep most of the important differences found in the original data.

PCA (Principal Component Analysis)

PCA is commonly used for data preprocessing for use with machine learning algorithms.

It helps to remove redundancy, improve computational efficiency and make data easier to visualize and analyze especially when dealing with high-dimensional data.

It helps you to reduce the number of features in a dataset while keeping the most important information.

It changes your original features into new features these new features don’t overlap with each other and the first few keep most of the important differences found in the original data.

How Principal Component Analysis Works?

PCA uses linear algebra to transform data into new features called principal components.

It finds these by calculating eigenvectors (directions) and eigenvalues (importance) from the covariance matrix.

PCA selects the top components with the highest eigenvalues and projects the data onto them simplify the dataset.

Imagine you’re looking at a messy cloud of data points like stars in the sky and want to simplify it.

PCA helps you find the "most important angles" to view this cloud so you don’t miss the big patterns.

Step 1: Standardize the Data

Different features may have different units and scales like salary vs. age. To compare them fairly PCA first standardizes the data by making each feature have:

- A mean of 0
- A standard deviation of 1

$$Z = \frac{X - \mu}{\sigma}$$

where:

- μ is the mean of independent features $\mu = \{\mu_1, \mu_2, \cdots, \mu_m\}$
- σ is the standard deviation of independent features $\sigma = \{\sigma_1, \sigma_2, \cdots, \sigma_m\}$

Step 2: Calculate Covariance Matrix

Next PCA calculates the **covariance matrix** to see how features relate to each other whether they increase or decrease together. The covariance between two features x_1 and x_2 is:

$$cov(x1, x2) = \frac{\sum_{i=1}^{n}(x1_i - \bar{x1})(x2_i - \bar{x2})}{n-1}$$

Where:

- $\bar{x}_1 \, and \, \bar{x}_2$ are the mean values of features $x_1 \, and \, x_2$
- n is the number of data points

The value of covariance can be positive, negative or zeros.

Step 3: Find the Principal Components

Principal Components in PCA

Principal Components (PCs) are new axes (directions) that represent the **spread (variance)** of the data.

1. **PC1 (1st Principal Component)**:
    - Direction of **maximum variance** in the data.
    - Captures the most information (variance) possible in a single dimension.
2. **PC2 (2nd Principal Component)**:
    - The next best direction **orthogonal (perpendicular)** to PC1.
    - Captures the next highest variance, and so on for PC3, PC4...

These Directions Come From

They are derived from:

- The **covariance matrix** of the data (let's call it matrix **A**).
- The **eigenvectors** and **eigenvalues** of matrix **A**.

These directions come from the **eigenvectors** of the covariance matrix and their importance is measured by **eigenvalues**. For a square matrix A an **eigenvector** X (a non-zero vector) and its corresponding **eigenvalue** λ satisfy:

$$AX=\lambda X$$

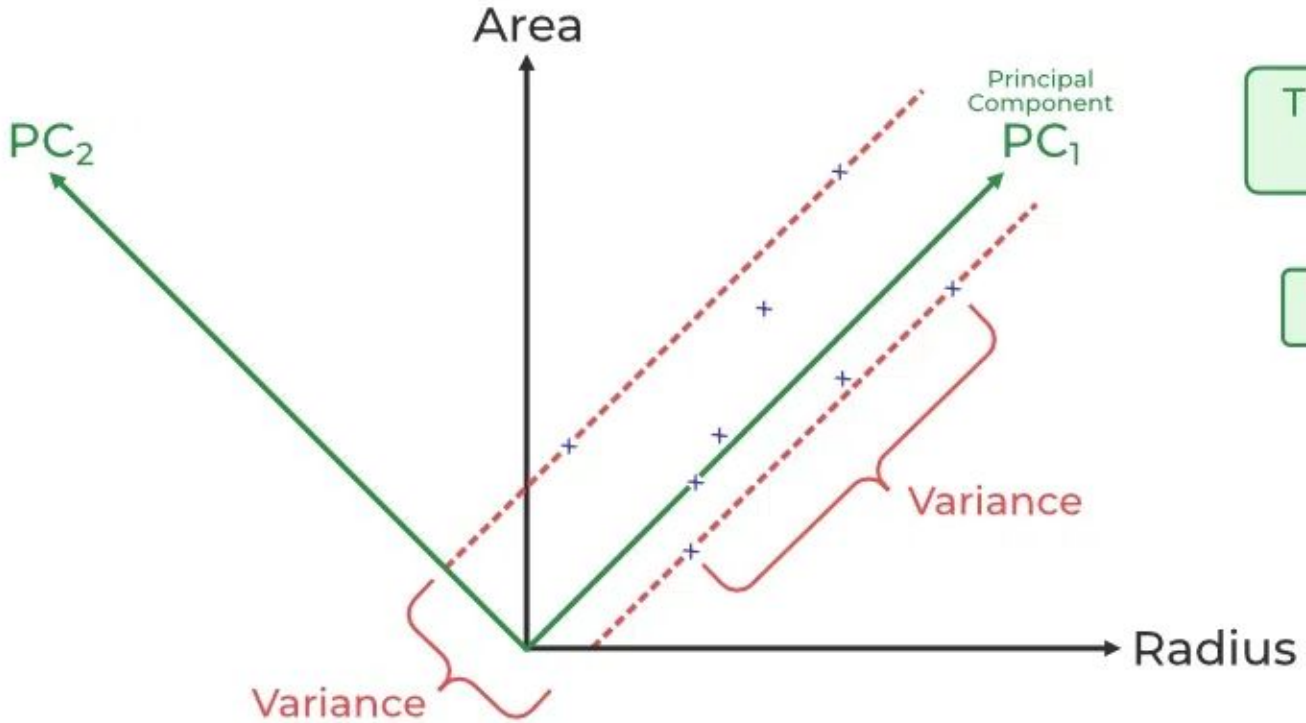
- **Eigenvectors** = Principal component **directions** (PC1, PC2, ...).
- **Eigenvalues** = How **important** each direction is (i.e., how much variance it explains).

Step 4: Pick the Top Directions & Transform Data

After calculating the eigenvalues and eigenvectors PCA ranks them by the amount of information they capture. We then:

1. **Select the top k components** hat capture most of the variance like 95%.
2. **Transform the original dataset** by projecting it onto these top components.

This means we reduce the number of features (dimensions) while keeping the important patterns in the data.



Transformation
2D → 1D

$PC_1 > PC_2$

In the above image the original dataset has two features "Radius" and "Area" represented by the black axes. PCA identifies two new directions: **PC_1** and **PC_2** which are the **principal components**.

- These new axes are rotated versions of the original ones. **PC_1** captures the maximum variance in the data meaning it holds the most information while **PC_2** captures the remaining variance and is perpendicular to PC_1.

- The spread of data is much wider along PC_1 than along PC_2. This is why PC_1 is chosen for dimensionality reduction. By projecting the data points (blue crosses) onto PC_1 we effectively **transform the 2D data into 1D and** retain most of the important structure and patterns.

Advantages of Principal Component Analysis

1. **Multicollinearity Handling:** Creates new, uncorrelated variables to address issues when original features are highly correlated.
2. **Noise Reduction:** Eliminates components with low variance enhance data clarity.
3. **Data Compression:** Represents data with fewer components reduce storage needs and speeding up processing.
4. **Outlier Detection:** Identifies unusual data points by showing which ones deviate significantly in the reduced space.

Disadvantages of Principal Component Analysis

1. **Interpretation Challenges:** The new components are combinations of original variables which can be hard to explain.
2. **Data Scaling Sensitivity:** Requires proper scaling of data before application or results may be misleading.
3. **Information Loss:** Reducing dimensions may lose some important information if too few components are kept.
4. **Assumption of Linearity:** Works best when relationships between variables are linear and may struggle with non-linear data.
5. **Computational Complexity:** Can be slow and resource-intensive on very large datasets.
6. **Risk of Overfitting:** Using too many components or working with a small dataset might lead to models that don't generalize well